Task 10: Build a Student CRUD application using Spring Boot. Map the database table using JPA annotations such as @Entity, @Id, @Table, and @Column. Perform Create, Read, Update, and Delete operations using a relational database.

Algorithm:

1. Start the Spring Boot application and load all required dependencies using Maven.
2. Configure MySQL database connection in application.properties.
3. Create a Student entity class and map it to the database using @Entity and @Table.
4. Define fields (id, name, email, course) and map them using @Id and @Column.
5. Create a StudentRepository interface extending JpaRepository for database operations.
6. Create a StudentService class to implement business logic.
7. Inject the repository into the service class using dependency injection.
8. Implement methods for Create, Read, Update, and Delete operations in the service layer.
9. Create a StudentController class and annotate it with @RestController.
10. Define base URL using @RequestMapping("/students").
11. Implement REST APIs using:
    - @PostMapping for Create
    - @GetMapping for Read
    - @PutMapping for Update
    - @DeleteMapping for Delete
12. Run the application using mvn spring-boot:run.
13. Test all CRUD operations using Postman or curl commands.
14. Verify the data stored in MySQL using SQL queries (SELECT * FROM students).
15. Stop the application after successful execution.

Programs

Home controller

```
package com.example.studentcrud.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        return "index";
    }
}
```

Entity

```
package com.example.studentcrud.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Column;
import jakarta.persistence.GeneratedValue;
```

```java
import jakarta.persistence.GenerationType;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "student_id")
    private Long id;

    @Column(name = "student_name", nullable = false, length = 100)
    private String name;

    @Column(name = "student_email", nullable = false, unique = true, length = 100)
    private String email;

    @Column(name = "student_course", nullable = false, length = 50)
    private String course;

    public Student() {
    }

    public Student(Long id, String name, String email, String course) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.course = course;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCourse() {
        return course;
    }
```

```
    public void setCourse(String course) {
        this.course = course;
    }
}
```

Repository

```
package com.example.studentcrud.repository;

import com.example.studentcrud.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

Student CURD

```
package com.example.studentcrud.service;

import com.example.studentcrud.entity.Student;
import com.example.studentcrud.repository.StudentRepository;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class StudentService {

    private final StudentRepository studentRepository;

    public StudentService(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    public Student createStudent(Student student) {
        return studentRepository.save(student);
    }

    public List<Student> getAllStudents() {
        return studentRepository.findAll();
    }

    public Student getStudentById(Long id) {
        Optional<Student> optionalStudent = studentRepository.findById(id);
        return optionalStudent.orElse(null);
    }

    public Student updateStudent(Long id, Student updatedStudent) {
        Optional<Student> optionalStudent = studentRepository.findById(id);

        if (optionalStudent.isPresent()) {
            Student existingStudent = optionalStudent.get();
            existingStudent.setName(updatedStudent.getName());
            existingStudent.setEmail(updatedStudent.getEmail());
```

```
            existingStudent.setCourse(updatedStudent.getCourse());
            return studentRepository.save(existingStudent);
        } else {
            return null;
        }
    }

    public String deleteStudent(Long id) {
        if (studentRepository.existsById(id)) {
            studentRepository.deleteById(id);
            return "Student deleted successfully";
        } else {
            return "Student not found";
        }
    }
}
```

Index.html

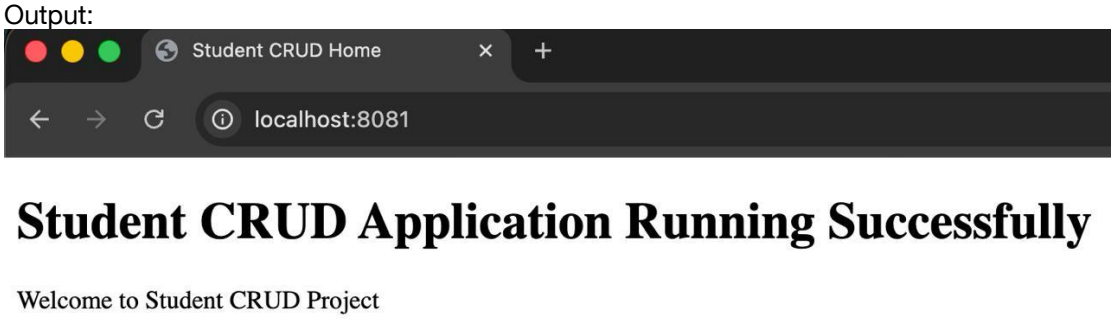
```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Student CRUD Home</title>
</head>
<body>
    <h1>Student CRUD Application Running Successfully</h1>
    <p>Welcome to Student CRUD Project</p>
</body>
</html>
```

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.3.5</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>studentcrud</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>studentcrud</name>
    <description>Student CRUD project</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
  </build>
</project>
```

Output:




```
Database changed
mysql> SELECT * FROM students;
+----+---------+-------------------------------+------------+------------+------------+-----+
| id | name    | email                         | dob        | department | phone      | age |
+----+---------+-------------------------------+------------+------------+------------+-----+
|  1 | krishna | krishnaprasath442006@gmail.com | 2026-04-08 | cse        | 9787164645 |   0 |
|  2 | krishna | krishnaprasath442006@gmail.com | 2026-04-08 | cse        | 9787164645 |   0 |
|  3 | arjun   | arjun123@gmail.com            | 2026-04-10 | IT         | 9787464632 |   0 |
|  4 | Krishna | NULL                          | NULL       | CSE        | NULL       |  21 |
|  5 | Krishna | NULL                          | NULL       | CSE        | NULL       |  21 |
|  6 | Arjun   | NULL                          | NULL       | IT         | NULL       |  22 |
|  7 | Priya   | NULL                          | NULL       | ECE        | NULL       |  20 |
|  8 | arunnn  | NULL                          | NULL       | IT         | NULL       |  22 |
|  9 | arunnn  | NULL                          | NULL       | IT         | NULL       |  22 |
+----+---------+-------------------------------+------------+------------+------------+-----+
9 rows in set (0.002 sec)
```

Result : thus task student crud has bencompleted successfully